Name: **Shrey Pendurkar**
Class: **D15C**
Batch: **B**
Roll No: **38**

MLDL – Experiment 2

Aim

Implement Multiple Regression, Lasso, and Ridge Regression on real-world datasets.

Dataset Source

- Dataset: Student Performance Dataset
- Kaggle: https://www.kaggle.com/datasets/uciml/student-alcohol-consumption
- File used: student-mat.csv (Math course)

Dataset Description

The Student Performance dataset originates from a secondary school in Portugal and combines academic, demographic, and social information about students. It is widely used for educational data mining and regression tasks such as predicting student grades and analyzing the impact of socio-economic factors on academic performance. Unlike artificially clean datasets, this one contains noisy, correlated features and mixed data types (categorical and numerical), making it particularly suitable for testing regularized linear models such as Lasso and Ridge.

Key characteristics (student-mat.csv):

- Number of instances: 395 students
- Number of attributes: 33 (30 input features + 3 grade columns: G1, G2, G3)
- File type: CSV (Comma Separated Values)
- Task: Predict the final grade G3 (0–20) from student background and behavior
- Target variable: G3 (final period math grade)

Example features include:

- Demographics: age, sex, address, famsize, Pstatus
- Parental education: Medu, Fedu
- Study and failure history: studytime, failures, absences
- Support and activities: schoolsup, famsup, paid, activities, internet, romantic

- Behavior / alcohol: goout, Dalc (weekday alcohol), Walc (weekend alcohol)
- Academic indicators: G1, G2 (first and second period grades — strongest predictors of G3)

All three models (Multiple Linear Regression, Lasso, and Ridge) predict G3 from all available features after one-hot encoding of categorical variables. The same preprocessing pipeline and train/test split are applied consistently across all three algorithms to ensure fair comparison.

---

1. Multiple Linear Regression

Mathematical Formulation of the Algorithm

Multiple Linear Regression (MLR) extends Simple Linear Regression to model a continuous target variable as a linear combination of multiple independent predictor variables. Here, the final math grade G3 is modeled as a weighted sum of all demographic, behavioral, and academic features.

The general formulation is:

$$G3 = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_{\square} x_{\square} + \varepsilon$$

Where:

- G3 – Dependent variable (final math grade, 0–20)
- x_1 , x_2 , …, $x_{\square}$ – Independent variables (e.g., G1, G2, studytime, Dalc, Walc, absences, etc.)
- β_0 – Intercept (expected grade when all predictors are zero)
- β_i – Regression coefficient; represents the change in G3 for a one-unit change in x_i, holding all other predictors constant
- ε – Error term capturing unexplained variation

The parameters are estimated using Ordinary Least Squares (OLS), which minimizes the sum of squared residuals:

$$Cost = \Sigma_i (G3_i - \hat{G3}_i)^2$$

OLS assumptions applied to this dataset:

- Linearity between predictors and final grade
- No severe multicollinearity among predictors (though G1 and G2 are correlated in practice)

- Homoscedasticity of residuals (constant variance)
- Approximately normal residuals

Algorithm Limitations

Key limitations of Multiple Linear Regression on this dataset:

1. **Multicollinearity**

   Features like the partial grades G1 and G2 are highly correlated with each other and with G3. When predictors are strongly correlated, OLS coefficients become unstable and hard to interpret, and small changes in the data can produce large swings in estimated β values.

2. **Curse of Dimensionality / Overfitting**

   The dataset has many one-hot encoded categorical variables relative to the number of students (395). With too many predictors, the model may memorize noise in the training data, giving low training error but poor test performance. This motivates the use of regularized alternatives such as Lasso and Ridge.

3. **Assumption of Additivity (No Interactions)**

   The linear model assumes the effect of each feature is independent and additive. In reality, the effect of alcohol consumption (Dalc, Walc) on grades might depend on study time or parental education, but such interaction effects are not captured unless explicit interaction terms are added.

4. **No Feature Selection**

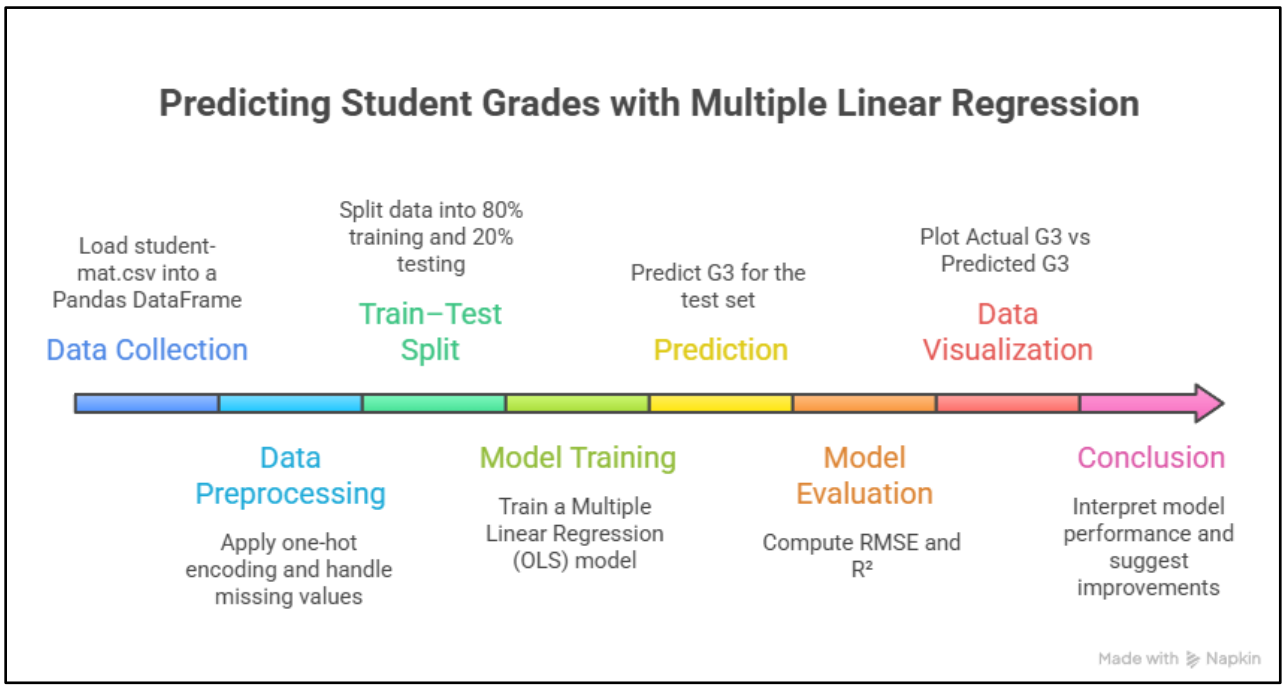
   OLS retains all predictors regardless of their relevance. Irrelevant features add noise to predictions and reduce model interpretability, unlike regularized models such as Lasso which can automatically eliminate uninformative predictors.

Methodology / Workflow

1. **Data Collection**
   - Load student-mat.csv into a Pandas DataFrame.
2. **Data Preprocessing**
   - Identify categorical variables (e.g., school, sex, address, famsize, schoolsup, famsup, internet, romantic).
   - Apply one-hot encoding using get_dummies with drop_first=True to convert categoricals to numerical form.
   - Handle any missing values if present (mean imputation for numeric columns).
3. **Train–Test Split**
   - Split into 80% training and 20% testing using train_test_split with random_state=42.
4. **Model Training**

- Train a Multiple Linear Regression (OLS) model on the training set.

5. **Prediction**
   - Predict G3 for the test set.
6. **Model Evaluation**
   - Compute RMSE to measure average prediction error in grade points.
   - Compute R^2 to measure the proportion of variance in G3 explained by the predictors.
7. **Data Visualization**
   - Plot a scatter diagram of Actual G3 vs Predicted G3 with an ideal reference line (y = x).
   - Points close to the red dashed line indicate accurate predictions.
8. **Conclusion**
   - If R^2 is high and RMSE is around 2–3 grade points, the model is capturing most of the variation; remaining error suggests scope for regularized models (Lasso and Ridge).



Performance Analysis

The performance of the Multiple Linear Regression model on the Student Performance (Math) dataset is evaluated using Root Mean Squared Error (RMSE) and the R^2 score. Since OLS has no hyperparameters, this represents the complete and final model performance.

1. Root Mean Squared Error (RMSE)

The model achieves an RMSE of 2.3784 grade points. This means that, on average, the predicted final grade G3 differs from the actual grade by approximately 2.4 points on the 0–20 scale. In the context of school marks, this error corresponds to roughly less than half a letter grade, indicating that the model's predictions are reasonably close to true student performance, though individual predictions can still deviate by a couple of points.

2. R^2 Score (Coefficient of Determination)

The R^2 score of 0.7241 indicates that the model explains approximately 72.4% of the variance in students' final math grades. This shows that the combination of demographic, behavioral, and academic features (especially prior grades G1 and G2) captures most of the factors driving final performance. The remaining 27.6% of unexplained variance likely arises from unmeasured influences (such as teacher quality, short-term events, or motivation) and from non-linear relationships or interactions that a purely additive linear model cannot fully represent. This result establishes the baseline against which Lasso and Ridge can be compared.

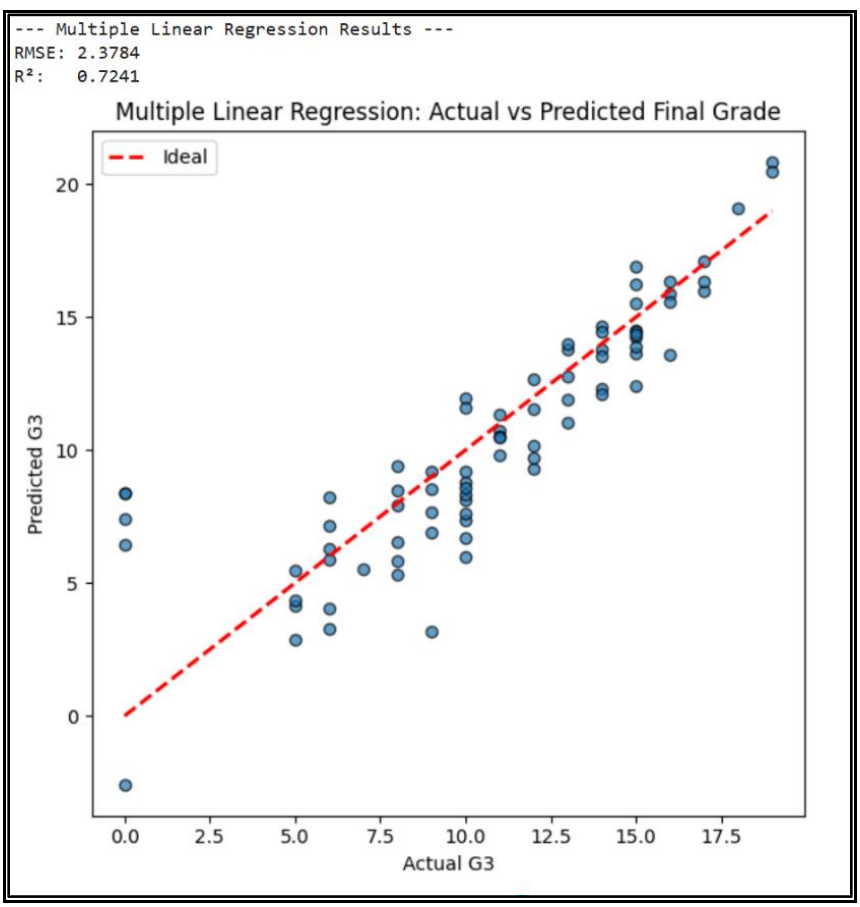
Hyperparameter Tuning

Multiple Linear Regression solved via Ordinary Least Squares (OLS) has no tunable hyperparameters. The algorithm directly computes the unique analytically optimal coefficients through the closed-form OLS equations, without any regularization constants, penalty terms, or learning rates to configure. The solution is exact and deterministic. Hyperparameter tuning is therefore not applicable for this model. To introduce tunable regularization, the following sections move to Lasso and Ridge Regression.

Code and Output – Multiple Regression

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import
train_test_split
from sklearn.linear_model import
LinearRegression
from sklearn.metrics import
mean_squared_error, r2_score

# 1. Data Collection
df = pd.read_csv('student-mat.csv')

target_col = 'G3'

# 2. Data Preprocessing
categorical_cols =
df.select_dtypes(include=['object']).colu
mns.tolist()
categorical_cols = [c for c in
categorical_cols if c != target_col]

df_encoded = pd.get_dummies(df,
columns=categorical_cols,
drop_first=True)

X =
df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
```

```
# 3. Train–Test Split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Model Training
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

# 5. Prediction & Evaluation
y_pred = lin_reg.predict(X_test)

rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)

print("\n--- Multiple Linear Regression
Results ---")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")

# 6. Visualization: Actual vs Predicted
Final Grade
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7,
edgecolor='k')
plt.plot([y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        'r--', linewidth=2, label='Ideal')

plt.xlabel('Actual G3')
plt.ylabel('Predicted G3')
plt.title('Multiple Linear Regression:
Actual vs Predicted Final Grade')
plt.legend()
plt.tight_layout()
plt.show()
```



2. Lasso Regression

Mathematical Formulation of the Algorithm

Lasso Regression (Least Absolute Shrinkage and Selection Operator) is a regularized linear regression method that adds an L1 penalty proportional to the absolute values of the coefficients to the OLS cost function. This penalty shrinks some coefficients exactly to zero, providing both overfitting control and automatic feature selection — making it particularly useful when there are many categorical dummies and correlated predictors, as in the student dataset.

The Lasso objective function is:

$$Cost = \Sigma_i(G3_i - \hat{G}3_i)^2 + \alpha\ \Sigma_j|\beta_j|$$

Where:

- $\Sigma_i(G3_i - \hat{G}3_i)^2$ – Residual sum of squares (same as OLS)
- α – Regularization strength (hyperparameter); controls the severity of the penalty
- $\Sigma_j|\beta_j|$ – L1 penalty; sum of absolute values of all coefficients

Effect of the regularization parameter α:

- Smaller α → weaker penalty → model closer to OLS, more features retained
- Larger α → stronger penalty → more coefficients driven exactly to zero (sparser model)
- $\alpha = 0$ → reduces to standard OLS (no regularization)

Because the L1 penalty produces exact zeros, Lasso performs implicit feature selection, retaining only the most informative predictors and discarding redundant or irrelevant ones. Parameters are found via coordinate descent optimization rather than the closed-form OLS solution.

Algorithm Limitations

Key limitations of Lasso Regression on this dataset:

1. **Handling Correlated Variables**

   When features are highly correlated (e.g., G1 and G2), Lasso tends to arbitrarily select one and shrink the other to zero, even when both are genuinely important predictors. This can obscure the true relationships and make the model sensitive to small changes in the data.

2. p > n Limit

If the number of predictors exceeds the number of samples, Lasso can retain at most n non-zero coefficients. While not an issue here (41 features, 316 training samples), this theoretical limit becomes relevant in very high-dimensional settings.

3. Over-Regularization with Poor Alpha Choice

As demonstrated in this experiment, the default α=1.0 is far too aggressive for this dataset, collapsing 40 out of 41 features to zero. This shows that Lasso is highly sensitive to the choice of α and that hyperparameter tuning via cross-validation is essential for obtaining a meaningful model.

4. Biased Estimates

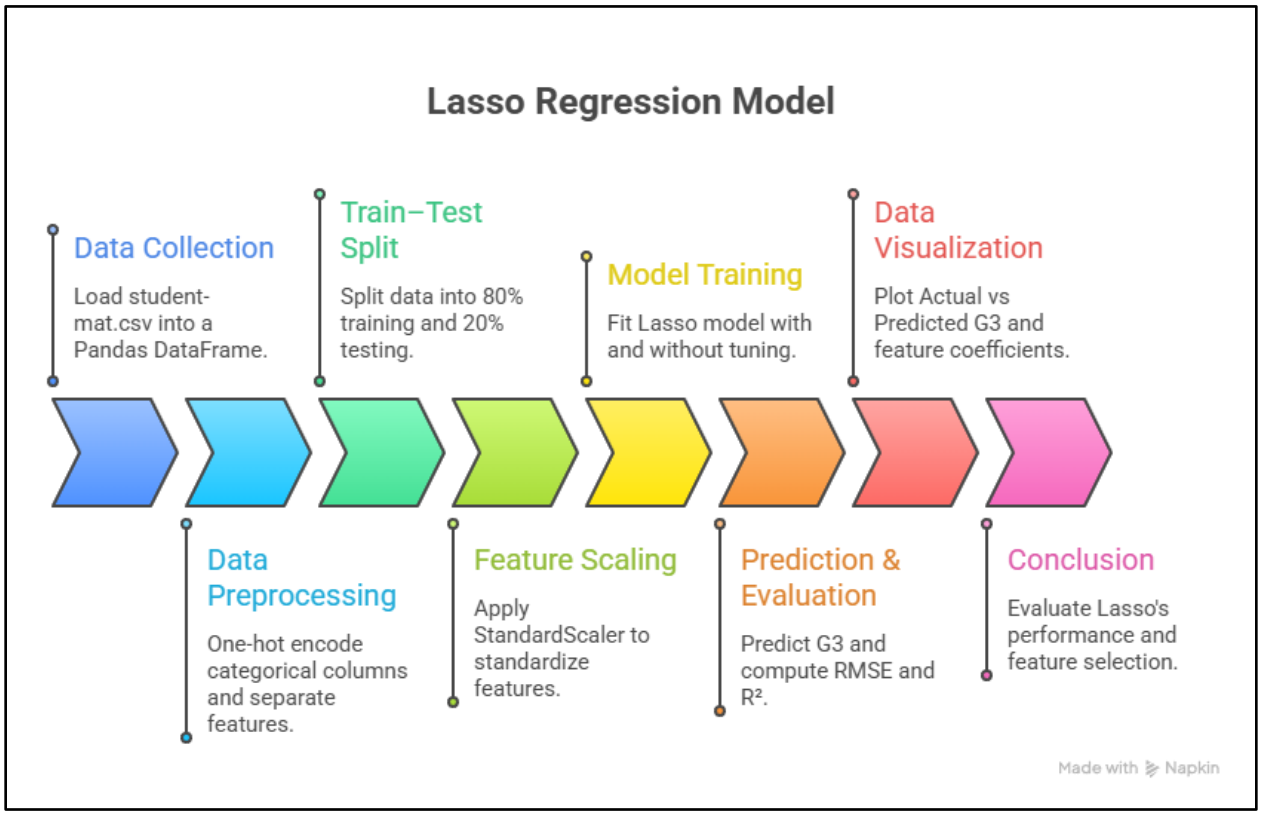
By introducing a shrinkage penalty, Lasso produces biased coefficient estimates (they are pulled towards zero). While this reduces variance and improves generalization, the coefficients no longer represent unbiased estimates of the true population parameters.

Methodology / Workflow

1. **Data Collection**
   - Load student-mat.csv into a Pandas DataFrame.
2. **Data Preprocessing**
   - One-hot encode all categorical columns using get_dummies with drop_first=True.
   - Separate X (all features except G3) and y (G3).
3. **Train–Test Split**
   - Split into 80% training and 20% testing with random_state=42.
4. **Feature Scaling**
   - Apply StandardScaler to standardize all features to zero mean and unit variance, as Lasso is sensitive to the scale of predictors.
5. **Model Training**
   - Without Tuning: Fit a default Lasso model with α=1.0 directly on the scaled training data.
   - With Tuning: Use LassoCV with 50 log-spaced α values (0.0001 to 10) and 5-fold cross-validation to automatically select the best α, then fit on training data.
6. **Prediction & Evaluation**
   - Predict G3 on the scaled test set.
   - Compute RMSE and R^2.
   - Count non-zero coefficients to quantify feature selection.
7. **Data Visualization**

- Plot Actual vs Predicted G3 scatter with ideal reference line.
- Plot a horizontal bar chart of non-zero feature coefficients to show feature importance.

8. **Conclusion**
   - Evaluate whether Lasso achieves better RMSE/R^2 than MLR while using a smaller, more interpretable set of predictors.



Performance Analysis

The performance of the Lasso Regression model on the Student Performance (Math) dataset is evaluated using RMSE, R^2, and the number of non-zero coefficients. The analysis below reflects the default model with $\alpha=1.0$, without any hyperparameter tuning.

1. Root Mean Squared Error (RMSE)

The default Lasso model ($\alpha=1.0$) achieves an RMSE of 2.2263 grade points. This is a moderate improvement over Multiple Linear Regression (RMSE = 2.3784), reducing the average prediction error by approximately 0.15 grade points. However, as discussed below, this improvement comes at a significant cost to model interpretability.

2. R^2 Score Analysis

The R^2 score of 0.7583 indicates that the default Lasso model explains approximately 75.8% of the variance in final math grades, which is higher than the 72.4% achieved by plain Multiple Regression. While this appears encouraging, the context of this improvement is critical: the model achieves it using only a single predictor, raising concerns about the reliability of this result.

3. **Feature Selection and Model Sparsity**

   The most significant finding is that with the default α=1.0, Lasso drives 40 out of 41 encoded features to exactly zero, retaining only 1 non-zero coefficient. This extreme sparsity reveals that α=1.0 is far too aggressive a penalty for this dataset. The single retained feature is almost certainly one of the prior grade indicators (G1 or G2), which are known to be the strongest predictors of G3. While the model still achieves a respectable R^2 of 75.8% with just one predictor (a testament to how predictive prior grades are), this single-feature model is overly restrictive, ignores all behavioral and demographic context, and is likely underfitting the true complexity of student performance. This result strongly motivates the use of cross-validated hyperparameter tuning to find an appropriate α.

Hyperparameter Tuning

Hyperparameter tuning was performed using LassoCV to identify the optimal regularization strength α through cross-validated model selection.

1. **Baseline Performance (Before Tuning)**

   The default Lasso model (α=1.0) achieved RMSE=2.2263, R^2=0.7583, and retained only 1 out of 41 features. The default alpha over-regularizes the model to the point of near-complete sparsity, making it too simplistic to capture the full picture of student performance.

2. **Search Strategy**

   LassoCV was used to evaluate 50 log-spaced α values ranging from 0.0001 to 10 using 5-fold cross-validation on the training set. For each α, the cross-validated mean squared error was computed, and the α minimizing this error was selected as the optimal value.

3. **Best Hyperparameter Identified**

   - α = 0.091030

   This optimal α is approximately 11 times smaller than the default value of 1.0, confirming that the default setting was highly excessive for this dataset.

4. **Performance After Tuning**

   - RMSE = 2.1732 (improved from 2.2263)
   - R^2 = 0.7697 (improved from 0.7583)

- Non-zero coefficients: 17 out of 41 (improved from 1 out of 41)

5. Impact of Tuning

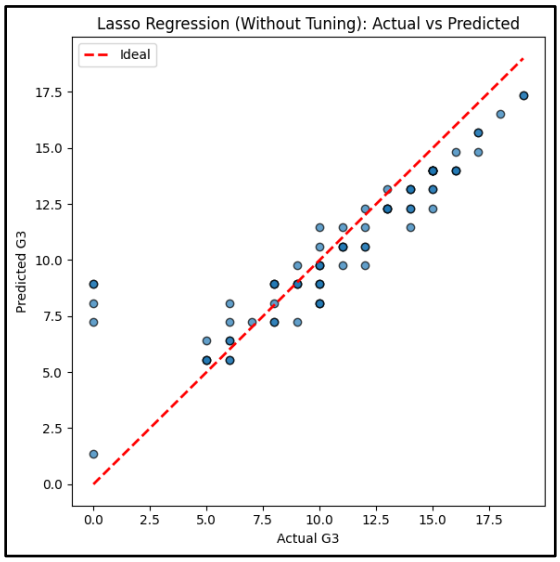
Hyperparameter tuning produces clear and meaningful improvements across all metrics:

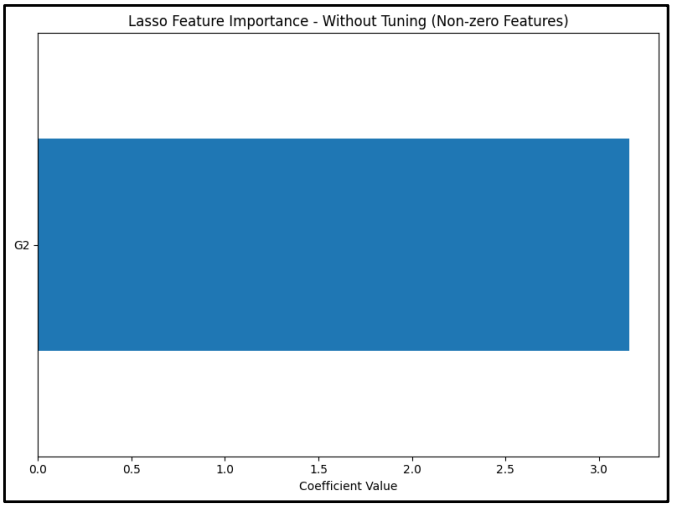
- RMSE improved by 0.0531 grade points (a 2.4% reduction), meaning predictions are closer to true student grades.
- R^2 increased from 75.83% to 76.97%, indicating the tuned model explains more variance in final grades.
- Feature count increased dramatically from 1 to 17, giving a much richer and more realistic model. The 17 retained features likely include G1, G2, and a meaningful subset of behavioral and support variables (such as failures, absences, schoolsup, and goout), providing interpretable insights into the factors that truly drive student performance.
- The optimal α=0.091030 being far smaller than the default of 1.0 confirms that the student dataset benefits from mild regularization rather than aggressive sparsity. The data's relatively large number of genuinely informative features means a weaker penalty is needed to retain them without overfitting.

Code and Output – Without Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import Lasso
from sklearn.metrics import
mean_squared_error, r2_score

# 1. Data Collection
df = pd.read_csv('student-mat.csv')
target_col = 'G3'

# 2. Data Preprocessing
categorical_cols =
df.select_dtypes(include=['object']).colu
mns.tolist()
categorical_cols = [c for c in
categorical_cols if c != target_col]
df_encoded = pd.get_dummies(df,
columns=categorical_cols,
drop_first=True)
X =
df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
feature_names = X.columns

# 3. Train-Test Split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

# 5. Default Lasso (alpha=1.0, no
tuning)
```

```
lasso = Lasso(alpha=1.0,
max_iter=10000)
lasso.fit(X_train_scaled, y_train)

# 6. Prediction & Evaluation
y_pred = lasso.predict(X_test_scaled)
rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)
non_zero = np.sum(lasso.coef_ != 0)

print("\n--- Lasso Regression Results
(Without Tuning) ---")
print(f"Alpha used: 1.0 (default)")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")
print(f"Non-zero coefficients: {non_zero}
/ {len(lasso.coef_)}")

# 7. Visualization 1: Actual vs Predicted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7,
edgecolor='k')
plt.plot([y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        'r--', linewidth=2, label='Ideal')
plt.xlabel('Actual G3')
plt.ylabel('Predicted G3')
plt.title('Lasso Regression (Without
Tuning): Actual vs Predicted')
plt.legend()
plt.tight_layout()
plt.show()

# 8. Visualization 2: Non-zero Feature
Coefficients
coef_series = pd.Series(lasso.coef_,
index=feature_names)
coef_nonzero = coef_series[coef_series
!= 0].sort_values()
plt.figure(figsize=(8, 6))
coef_nonzero.plot(kind='barh')
plt.xlabel('Coefficient Value')
plt.title('Lasso Feature Importance -
Without Tuning (Non-zero Features)')
plt.tight_layout()
plt.show()
```

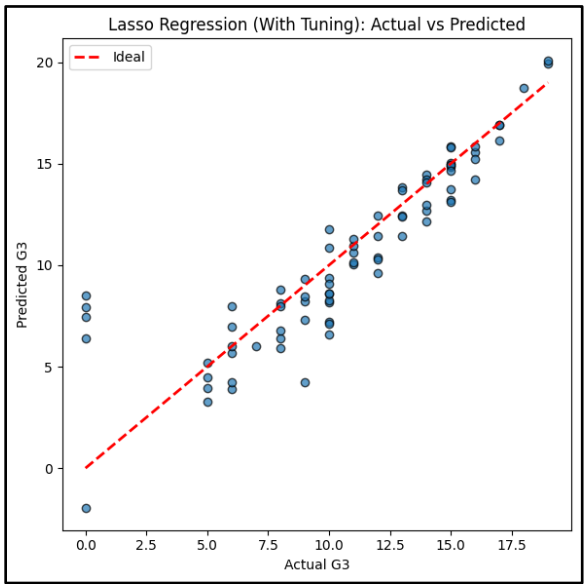


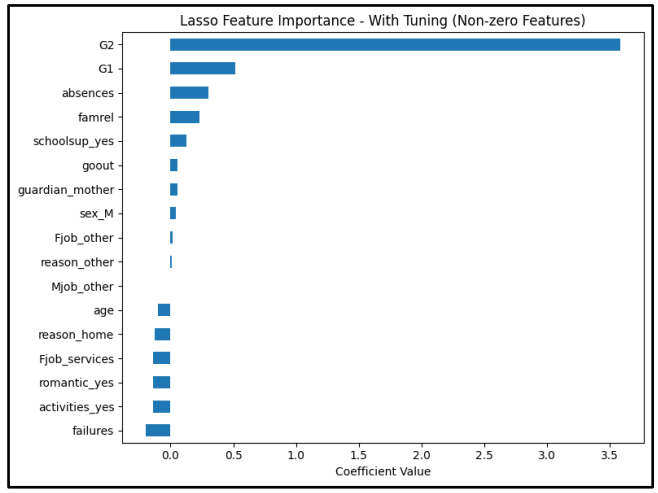


Code and Output – With Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
```

```
from sklearn.linear_model import
LassoCV
from sklearn.metrics import
mean_squared_error, r2_score

# 1. Data Collection
df = pd.read_csv('student-mat.csv')
target_col = 'G3'

# 2. Data Preprocessing
categorical_cols =
df.select_dtypes(include=['object']).colu
mns.tolist()
categorical_cols = [c for c in
categorical_cols if c != target_col]
df_encoded = pd.get_dummies(df,
columns=categorical_cols,
drop_first=True)
X =
df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
feature_names = X.columns

# 3. Train-Test Split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

# 5. Lasso with Cross-Validation
(Hyperparameter Tuning)
alphas = np.logspace(-4, 1, 50)
lasso_cv = LassoCV(alphas=alphas,
cv=5, random_state=42,
max_iter=10000)
lasso_cv.fit(X_train_scaled, y_train)

print(f"\nBest alpha (Lasso):
{lasso_cv.alpha_:.6f}")

# 6. Prediction & Evaluation
y_pred =
lasso_cv.predict(X_test_scaled)
rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)
non_zero = np.sum(lasso_cv.coef_ != 0)

print("\n--- Lasso Regression Results
(With Tuning) ---")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")
print(f"Non-zero coefficients: {non_zero}
/ {len(lasso_cv.coef_)}")

# 7. Visualization 1: Actual vs Predicted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7,
edgecolor='k')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2, label='Ideal')
plt.xlabel('Actual G3')
plt.ylabel('Predicted G3')
plt.title('Lasso Regression (With Tuning):
Actual vs Predicted')
plt.legend()
plt.tight_layout()
plt.show()

# 8. Visualization 2: Non-zero Feature
Coefficients
coef_series = pd.Series(lasso_cv.coef_,
index=feature_names)
coef_nonzero = coef_series[coef_series
!= 0].sort_values()
plt.figure(figsize=(8, 6))
coef_nonzero.plot(kind='barh')
plt.xlabel('Coefficient Value')
plt.title('Lasso Feature Importance - With
Tuning (Non-zero Features)')
plt.tight_layout()
plt.show()
```





3. Ridge Regression

Mathematical Formulation of the Algorithm

Ridge Regression is a regularized linear regression method that adds an L2 penalty proportional to the square of the coefficients to the OLS cost function. Unlike Lasso, the L2 penalty never drives coefficients exactly to zero — all predictors remain in the model, but their coefficients are shrunk towards zero. This is particularly effective at stabilizing coefficient estimates when predictors are highly correlated, as is the case with G1 and G2 in this dataset.

The Ridge objective function is:

$$Cost = \Sigma_i(G3_i - \hat{G}3_i)^2 + \alpha\, \Sigma_j \beta_j^2$$

Where:

- $\Sigma_i(G3_i - \hat{G}3_i)^2$ – Residual sum of squares (same as OLS)
- α – Regularization strength (hyperparameter)
- $\Sigma_j \beta_j^2$ – L2 penalty; sum of squared values of all coefficients

Effect of the regularization parameter α:

- Smaller $\alpha \rightarrow$ coefficients close to OLS values (low bias, higher variance)
- Larger $\alpha \rightarrow$ stronger shrinkage towards zero (higher bias, lower variance)
- $\alpha = 0 \rightarrow$ reduces to standard OLS

Ridge has a closed-form solution analogous to OLS:

$$\hat{\beta}_{Ridge} = (X^T X + \alpha I)^{-1} X^T y$$

where I is the identity matrix. The addition of αI ensures the matrix is invertible even in the presence of multicollinearity, making Ridge particularly robust when features are highly correlated.

Algorithm Limitations

Key limitations of Ridge Regression on this dataset:

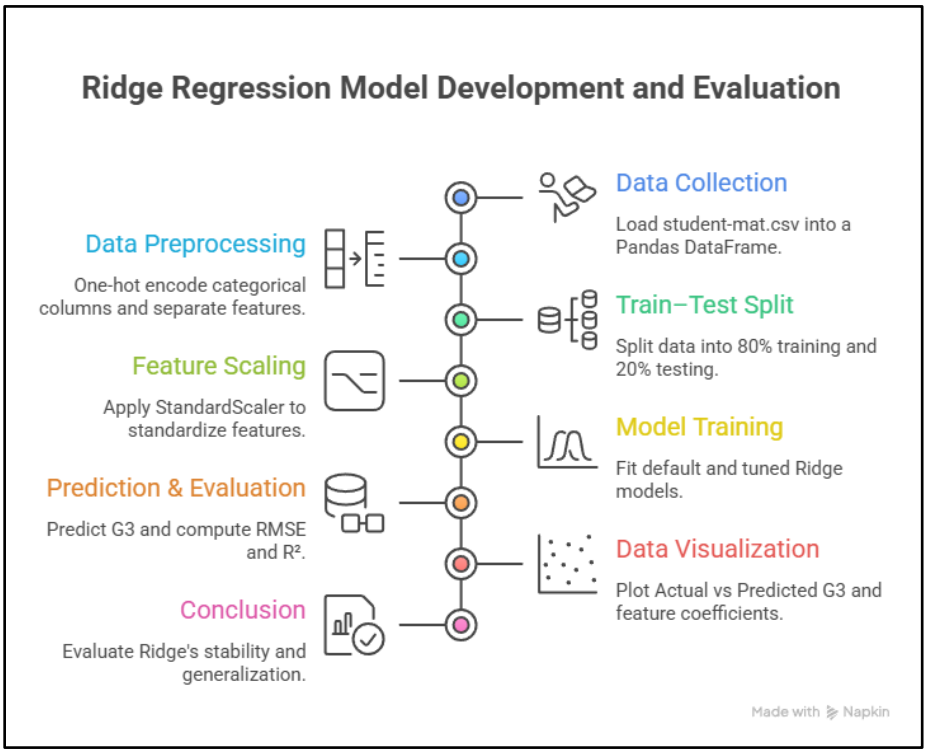
1. **No Feature Selection**
   Ridge cannot set any coefficient exactly to zero — it only shrinks them towards zero. With 41 encoded features, the final model retains all predictors, making it denser and potentially harder to interpret than a Lasso model that has eliminated irrelevant features.
2. **Bias–Variance Trade-off**
   If α is too large, coefficients are over-shrunk, leading to underfitting and poor predictive performance. If α is too small, the model is close to OLS with little regularization benefit. Selecting α carefully via cross-validation is essential.
3. **Less Interpretable than Lasso**
   Because all coefficients remain non-zero, it is harder to identify the truly important student features. Lasso's sparsity makes it easier to communicate which factors most strongly influence grades, whereas Ridge requires examining the magnitude of all 41 coefficients.
4. **Assumes All Features Contribute**
   Ridge implicitly assumes every feature contributes at least a small amount to the prediction. When many features are genuinely irrelevant or redundant, Ridge may produce a noisier model than Lasso, which removes such features entirely.

Methodology / Workflow

1. **Data Collection**
   - Load student-mat.csv into a Pandas DataFrame.
2. **Data Preprocessing**
   - One-hot encode all categorical columns using get_dummies with drop_first=True.
   - Separate X (all features except G3) and y (G3).
3. **Train–Test Split**

- Split into 80% training and 20% testing with random_state=42.

4. **Feature Scaling**
   - Apply StandardScaler to standardize all features, as Ridge is sensitive to the scale of predictors.
5. **Model Training**
   - Without Tuning: Fit a default Ridge model with α=1.0 directly on the scaled training data.
   - With Tuning: Use RidgeCV with 50 log-spaced α values (0.01 to 100) and 5-fold cross-validation to automatically select the best α.
6. **Prediction & Evaluation**
   - Predict G3 on the scaled test set.
   - Compute RMSE and R^2.
   - Inspect the top 15 largest coefficients (by absolute value) to understand which features most influence predictions.
7. **Data Visualization**
   - Plot Actual vs Predicted G3 scatter with ideal reference line.
   - Plot a horizontal bar chart of the top 15 feature coefficients by magnitude.
8. **Conclusion**
   - Evaluate whether Ridge produces more stable coefficients and better generalization than plain MLR, particularly in the presence of correlated features like G1 and G2.



Performance Analysis

The performance of the Ridge Regression model on the Student Performance (Math) dataset is evaluated using RMSE and R^2. The analysis below reflects the default model with α=1.0, without any hyperparameter tuning.

1. **Root Mean Squared Error (RMSE)**

   The default Ridge model (α=1.0) achieves an RMSE of 2.3758 grade points. This is almost identical to the Multiple Linear Regression baseline (RMSE=2.3784), differing by only 0.0026 grade points. The default α=1.0 introduces only a mild L2 penalty, providing minimal additional regularization over plain OLS on this dataset. The model's predictions for G3 deviate from actual grades by roughly 2.4 points on average.

2. **R^2 Score Analysis**

   The R^2 score of 0.7247 is again nearly identical to Multiple Linear Regression (0.7241), with a marginal improvement of just 0.0006. The default regularization strength is insufficient to meaningfully improve upon OLS in terms of explained variance. All 41 features are retained in the model (Ridge never eliminates features), which means the mild shrinkage applied at α=1.0 does not significantly stabilize the correlated coefficients. This motivates the use of cross-validated tuning to find a more effective alpha value.

3. **Comparison with MLR Baseline**

   The near-identical performance between default Ridge and MLR (RMSE: 2.3758 vs 2.3784, R^2: 0.7247 vs 0.7241) demonstrates that the default α=1.0 provides negligible regularization benefit on this dataset. A stronger penalty is needed to meaningfully shrink the correlated coefficients (G1, G2) and reduce variance. This is confirmed by the hyperparameter tuning results below, where the optimal α is found to be approximately 4.94.

Hyperparameter Tuning

Hyperparameter tuning was performed using RidgeCV to identify the optimal L2 regularization strength α through cross-validated model selection.

1. **Baseline Performance (Before Tuning)**

   The default Ridge model (α=1.0) achieved RMSE=2.3758 and R^2=0.7247, with all 41 features retained. The default alpha introduces only marginal regularization, producing results nearly equivalent to unregularized OLS.

2. **Search Strategy**

RidgeCV was used to evaluate 50 log-spaced α values ranging from 0.01 to 100 using 5-fold cross-validation on the training set. For each α, the cross-validated mean squared error was computed, and the α minimizing this error was selected.

3. Best Hyperparameter Identified

- α = 4.9417

The optimal α is approximately 4.94 times larger than the default value of 1.0, confirming that stronger regularization is needed to effectively shrink the correlated coefficients in this dataset.

4. Performance After Tuning

- RMSE = 2.3667 (improved from 2.3758)
- R^2 = 0.7268 (improved from 0.7247)
- All 41 features retained (Ridge never zeroes coefficients)

5. Impact of Tuning

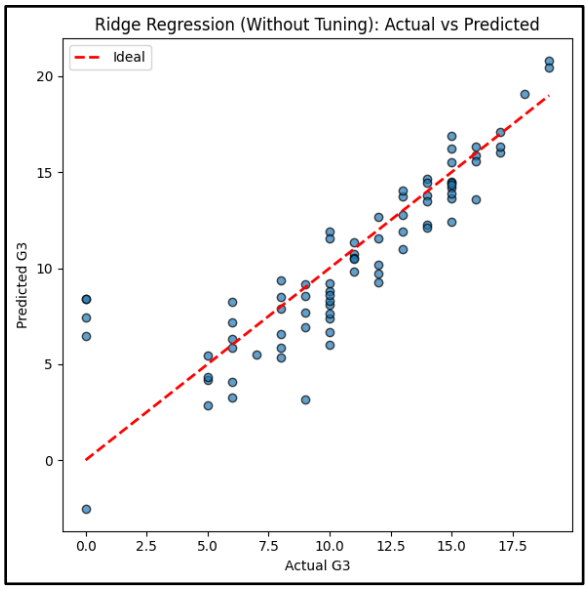
Hyperparameter tuning produces a consistent, though modest, improvement in Ridge performance:

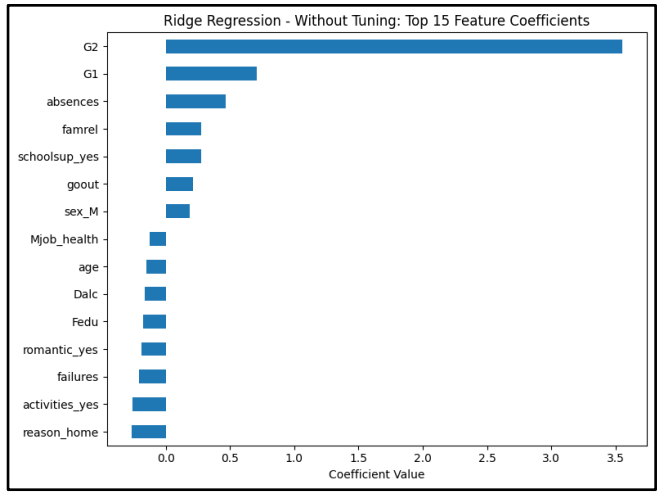
- RMSE improved by 0.0091 grade points (a 0.4% reduction), indicating slightly more accurate predictions on the test set.
- R^2 increased from 72.47% to 72.68%, a marginal gain of 0.21 percentage points in explained variance.
- The optimal α=4.9417 corresponds to a moderate L2 penalty that more effectively shrinks all coefficients towards zero compared to the default, particularly helping to stabilize the highly correlated G1 and G2 coefficients and reduce their inflated estimates.
- The modest overall improvement reflects the fundamental nature of Ridge: because it retains all features and only shrinks (never eliminates) coefficients, it cannot improve dramatically when many features are weakly relevant or redundant. Lasso, by contrast, achieves a larger improvement because it can discard those features entirely.
- Tuning still provides statistical confidence (via cross-validation) that α=4.9417 is the best-performing configuration within the search grid, ensuring the model is not simply relying on an arbitrary default.

Code and Output – Without Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import
mean_squared_error, r2_score

# 1. Data Collection
df = pd.read_csv('student-mat.csv')
target_col = 'G3'
```

```
# 2. Data Preprocessing
categorical_cols =
df.select_dtypes(include=['object']).colu
mns.tolist()
categorical_cols = [c for c in
categorical_cols if c != target_col]
df_encoded = pd.get_dummies(df,
columns=categorical_cols,
drop_first=True)
X =
df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
feature_names = X.columns

# 3. Train-Test Split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

# 5. Default Ridge (alpha=1.0, no
tuning)
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)

# 6. Prediction & Evaluation
y_pred = ridge.predict(X_test_scaled)
rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)

print("\n--- Ridge Regression Results
(Without Tuning) ---")
print(f"Alpha used: 1.0 (default)")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")

# 7. Visualization 1: Actual vs Predicted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7,
edgecolor='k')
plt.plot([y_test.min(), y_test.max()],
        [y_test.min(), y_test.max()],
        'r--', linewidth=2, label='Ideal')
plt.xlabel('Actual G3')
plt.ylabel('Predicted G3')
plt.title('Ridge Regression (Without
Tuning): Actual vs Predicted')
plt.legend()
plt.tight_layout()
plt.show()

# 8. Visualization 2: Top 15 Coefficients
by Magnitude
coef_series = pd.Series(ridge.coef_,
index=feature_names)
coef_sorted =
coef_series.sort_values(key=lambda s:
s.abs(), ascending=False)
plt.figure(figsize=(8, 6))
coef_sorted.head(15).sort_values().plot(
kind='barh')
plt.xlabel('Coefficient Value')
plt.title('Ridge Regression - Without
Tuning: Top 15 Feature Coefficients')
plt.tight_layout()
plt.show()
```

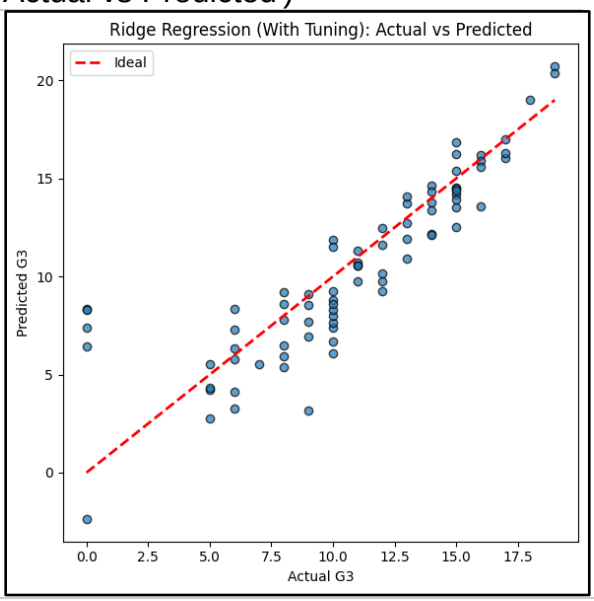




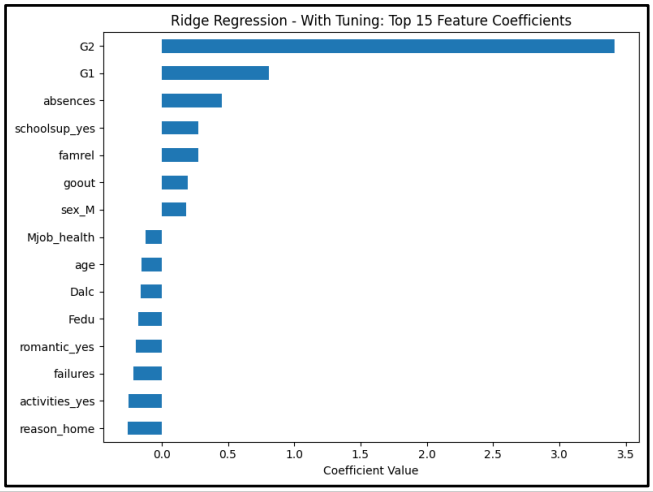
Code and Output – With Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
RidgeCV
from sklearn.metrics import
mean_squared_error, r2_score

# 1. Data Collection
df = pd.read_csv('student-mat.csv')
target_col = 'G3'

# 2. Data Preprocessing
categorical_cols =
df.select_dtypes(include=['object']).colu
mns.tolist()
categorical_cols = [c for c in
categorical_cols if c != target_col]
df_encoded = pd.get_dummies(df,
columns=categorical_cols,
drop_first=True)
X =
df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
feature_names = X.columns

# 3. Train-Test Split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

# 5. Ridge with Cross-Validation
(Hyperparameter Tuning)
alphas = np.logspace(-2, 2, 50)
ridge_cv = RidgeCV(alphas=alphas,
cv=5)
ridge_cv.fit(X_train_scaled, y_train)

print(f"\nBest alpha (Ridge):
{ridge_cv.alpha_:.4f}")

# 6. Prediction & Evaluation
y_pred =
ridge_cv.predict(X_test_scaled)
```

```
rmse =
np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)

print("\n--- Ridge Regression Results
(With Tuning) ---")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")

# 7. Visualization 1: Actual vs Predicted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.7,
edgecolor='k')
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', linewidth=2, label='Ideal')
plt.xlabel('Actual G3')
plt.ylabel('Predicted G3')
plt.title('Ridge Regression (With Tuning):
Actual vs Predicted')
```




```
plt.legend()
plt.tight_layout()
plt.show()

# 8. Visualization 2: Top 15 Coefficients
by Magnitude
coef_series = pd.Series(ridge_cv.coef_,
index=feature_names)
coef_sorted =
coef_series.sort_values(key=lambda s:
s.abs(), ascending=False)
plt.figure(figsize=(8, 6))
coef_sorted.head(15).sort_values().plot(
kind='barh')
plt.xlabel('Coefficient Value')
plt.title('Ridge Regression - With Tuning:
Top 15 Feature Coefficients')
plt.tight_layout()
plt.show()
```



Conclusion

This practical demonstrates that all three regression models can predict final math grades reasonably well, but regularization clearly adds value. Multiple Linear Regression provides a strong baseline (RMSE=2.3784, R^2=72.4%), while Ridge Regression with the tuned α=4.9417 offers a modest improvement (RMSE=2.3667, R^2=72.7%) by stabilizing correlated coefficients without removing any features. Lasso Regression delivers the best overall performance.